M26-CS 1.304
Data Structures and Algorithms for Problem Solving
Assignment–3

Important Points

- Only C++ is allowed.
- Use only the allowed headers mentioned in the question you are given in the lab.
- This is an in-lab assignment. The lab will be held on **2nd November 2026**.
- The lab starts at **8:30 AM** as usual and will be of **3 hours** duration. Please be on time.
- There are three questions listed in this document. In the lab, each student will be given **one** of the three questions to solve.
- The topics listed under each question are the topics you need to prepare. The questions asked in the lab will be based on these listed topics.
- You are expected to prepare **all three** questions, since you will not know in advance which one you will get.
- If you have any doubts please post them on Moodle.
- You are not allowed to use STL or any inbuilt function for the algorithms until explicitly mentioned in the question.

---

1 Graph / Tree

Problem Statement: Prepare the following graph and tree techniques. You should be comfortable with implementing each of them from scratch and applying them to solve problems on graphs and trees.

Topics

1. **Bridges and Articulation Points**
   - Finding all bridges (cut edges) in an undirected graph.
   - Finding all articulation points (cut vertices) in an undirected graph.
   - Discovery time and low-link values using DFS.
2. **Disjoint Set Union (DSU)**
   - Union and find operations.
   - Path compression and Union by rank/size.
   - Applications such as connectivity queries
3. **Euler Tour**
   - Euler tour of a tree (entry and exit times).

- Subtree queries using the Euler tour order. (No questions requiring segment tree will be asked for this part).

4. **Binary Lifting**
   - Preprocessing the 2^k-th ancestors of every node.
   - k-th ancestor queries.
   - Lowest Common Ancestor (LCA) of two nodes.

Expected Time Complexity

- Bridges / Articulation Points: $O(V + E)$
- DSU (per operation): $O(\alpha(N))$ amortized
- Euler Tour: $O(V)$
- Binary Lifting: $O(N \log N)$ preprocessing, $O(\log N)$ per query

Note

1. **Allowed:** `vector` and `map` (and other basic containers) can be used to store the graph, adjacency lists, and related data.
2. **Not allowed:** Inbuilt or library implementations of the algorithms in the topics. Bridges, articulation points, DSU, Euler tour and binary lifting must be implemented by you from scratch.

2 Big Integer

Problem Statement: Create a big integer library, similar to the one available in Java. The library should be able to store arbitrarily large integers and perform basic math operations, evaluate arithmetic expressions, and compare big integers.

Topics

1. **Expression Evaluation**
   - Operators: addition (+), subtraction (−), multiplication (×) and division (/).
   - Evaluate an expression following the BODMAS rule.
     - Example: Given the expression `3+6-5/5`, evaluate it using the BODMAS rule.
   - Convert the expression to postfix or prefix form and evaluate it (see postfix and prefix notation). No need to consider negative answers, questions ensure that no negative values occur while evaluaion.
2. **Comparison Operations**
   - Less than (<)
   - Greater than (>)
   - Equal to (=)

Expected Time Complexity

- Addition ($s_1 + s_2$): $O(n + m)$
- Subtraction ($s_1 - s_2$): $O(n + m)$
- Multiplication ($s_1 \times s_2$): $O(n \cdot m)$
- Division (s_1 / s_2): $O(n \cdot m)$
- Comparison: $O(n + m)$

Note

1. **Allowed:** `stack` can be used for expression evaluation (infix to postfix conversion and postfix evaluation). The `string` library and its methods are also allowed.
2. **Not allowed:** Any inbuilt big integer library or type. The big integer arithmetic, comparison and expression evaluation logic must be implemented by you from scratch.

3 Number Theory

Problem Statement: Prepare the following number theory topics. You should know the theory behind each of them as well as how to implement them efficiently.

Topics

1. **Sieve of Eratosthenes**.
2. **Prime Factorisation** in $O(\log n)$ per query using the sieve.
3. **Modular Inverse**.
4. **Square Root** computation in $O(\log n)$.
5. **Binary Exponentiation**.
6. **Chinese Remainder Theorem (CRT)**.
7. nCr**,** nPr **and Factorial** (including computation under a modulus).
8. **GCD / LCM**.

Expected Time Complexity

- Sieve: $O(n \log(\log(n)))$
- Prime Factorisation (after sieve): $O(\log n)$ per query
- Square Root: $O(\log n)$
- Binary Exponentiation: $O(\log n)$
- GCD: $O(\log(\min(a, b)))$

Note

1. **Allowed:** Basic containers such as `vector` can be used (for example, to store the sieve and primes).
2. **Not allowed:** Inbuilt functions such as `__gcd`, `gcd`, `lcm`, `pow`, `sqrt`, etc. You must write your own implementations of the sieve, factorisation, modular inverse, square root, binary exponentiation, CRT, nCr/nPr/factorial and GCD/LCM.

All the best!